Methodology & Tools Specification

> **Submission Deliverable:**
> In accordance with the MoveInSync assessment requirements, this document provides the technical methodology explaining how reviews were extracted and triaged, alongside a detailed breakdown of the tools selected across each layer of the pipeline.

Part 1: Engineering Methodology (How We Got There)

1. Problem Formulation & The FDE Approach

The MoveInSync assignment explicitly notes:

> *"How are our Play Store ratings? Work out why the ratings are where they are and tell us exactly what you would do to raise them... This is a question we'll keep asking from time to time. It is not a one-time ask."*

Manual review inspection introduces human selection bias, cannot quantify failure proportions, and incurs recurring overhead every time leadership requests an update. As a Forward Deployed Engineer (FDE), the methodology centers on replacing manual triage with an **automated, repeatable, in-memory data pipeline** that connects public user feedback directly to internal engineering workflows.

---

2. Live Data Ingestion & Ephemeral Stream Methodology

- **Unbiased API Sampling:** Rather than relying on static fixtures or manual CSV exports, the methodology queries Google Play's live backend API for `com.moveinsync.ets` across the 1,000 most recent reviews sorted by `Sort.NEWEST`. This captures immediate user sentiment following recent mobile releases.
- **In-Memory Processing:** Standard pipelines suffer performance degradations from intermediate disk writes. The methodology ingests raw API JSON directly into an in-memory `pandas.DataFrame`. All transformations—sentiment segregation, keyword tagging, vector embedding, and cohort partitioning—occur strictly in RAM. Disk writes are restricted solely to final output serialization (`output/app_health_report.html`, `output/mock_jira_tickets.json`, `output/rating_trend_history.json`).

---

3. NLP Triage & Multi-Label Taxonomy Methodology

Negative reviews (1 and 2 stars) are isolated as the statistical anchor driving ratings down. Because commuters frequently report compounding issues (e.g., "*app crashed while cab was delayed*"), the methodology implements a multi-label classification taxonomy:

- **GPS / Location Tracking:** Identifies tracking lag, marker freezing, and location jumps.
- **Authentication / OTP:** Identifies session expirations and SMS delivery queue bottlenecks.
- **App Stability & Performance:** Identifies ANR dialogs, crash traces, and UI freezes.
- **Booking & Roster Management:** Identifies race conditions and lockups at shift cutoffs.

---

4. RAG-Grounded Data Quality & Verification Methodology

Public Play Store reviews are inherently noisy, containing bot spam, 1-word non-actionable complaints ("*worst*"), or complaints about unrelated services.

To ensure engineering teams receive only genuine, actionable bugs, the methodology applies a **5-tier verification funnel**:

1. **Feature Knowledge Base Ingestion:** Live-crawls MoveInSync product documentation and Play Store feature specifications, creating an in-memory semantic grounding base.
2. **Heuristic Pre-Filter:** Rejects entries under 5 words (lacking reproduction context) and sentiment mismatches (1-star reviews stating "*great app*").
3. **ChromaDB Cosine Relevance Cutoff:** Embeds incoming reviews and calculates cosine distance to the MoveInSync feature corpus ($0.0 = identical, 2.0 = opposite$). Complaints with distance > 0.65 are discarded as off-topic.
4. **LLM Verification Audit:** Borderline complaints ($0.48 < dist ≤ 0.65$) are passed to Google GenAI (`gemini-2.5-flash`) with a single-token binary audit (`REAL` vs `FAKE`) to verify software defect legitimacy.
5. **Fail-Open Quota Protection:** API calls are capped at 40 per execution, gracefully failing open to prevent review loss under rate limits.

---

5. Semantic Deduplication & Jira Synthesis Methodology

Creating individual tickets for every negative review causes sprint backlog paralysis. When 50 users complain about map freezes, engineers require **one consolidated ticket with total impact metrics**.

The methodology uses **Greedy Cosine Clustering**:

1. Normalized review vectors v_i, v_j are compared via dot product: $Sim(v_i, v_j) = v_i \cdot v_j$.
2. Reviews exceeding a cosine similarity threshold of 0.58 are merged into a single bug entity.
3. Each cluster calculates:

- `affected_users` : Total count of commuters impacted by this specific bug.
- `sample_reviews` : Up to 3 verbatim user quotes providing reproduction context.
- `max_severity` : The lowest star rating in the cluster (escalating priority to P0/P1).
- `rag_relevance_score` : Proximity score to official feature specifications.

---

6. Longitudinal Trajectory & Snapshot Methodology

Standard scrapers extract reviews at a single point in time, creating point-in-time amnesia where teams cannot measure whether an app update raised ratings.

The methodology addresses this with two mechanisms:

1. **Intra-Dataset Weekly Cohort Partitioning:** Parses review submission timestamps (`at`) into calendar weeks (`df['review_date'].dt.to_period('W')`), isolating the 6 most recent weeks and computing week-over-week (WoW) deltas: $\Delta \text{ WoW} = \text{Rating}_{W_k} - \text{Rating}_{W_{k-1}}$.
2. **Persistent Run Snapshot Retention:** Every run appends an idempotent execution record to `output/rating_trend_history.json` , enabling multi-month comparison across releases.

Part 2: Tools Selected & Implementation Rationale (Which Tools We Used)

The toolchain was selected to maximize reliability, eliminate vendor lock-in, and maintain a $0 operational cost:

Tool / Library	Function in Pipeline	Selection Rationale
`google-play-scraper` **(v1.2.4)**	Live review extraction without browser automation	Queries Play Store API directly; zero credentials, sub-second execution
`pandas` **(v2.0+)**	In-memory data manipulation & datetime cohort grouping	Vectorized text operations; fast in-memory execution and cohort filtering
`requests` + `bs4` **(BeautifulSoup4)**	Knowledge base web crawler & HTML document cleaner	Live feature extraction from public MoveInSync product web pages
`sentence-transformers` (`all-MiniLM-L6-v2`)	Local 384-dim dense neural vector embedding generation	CPU-optimized embeddings; 0 latency, $0 API operational cost
`chromadb` **(In-Memory Client)**	Vector store with HNSW cosine distance indexing	In-memory client; no server or Docker host required
`google-genai` (`gemini-2.5-flash`)	High-throughput LLM audit for ambiguous reviews	Fast binary validation; free-tier quota friendly
Atlassian Jira API (REST v2 schema)	Issue schema serialization into `mock_jira_tickets.json`	Standard bulk REST API format; ready for direct sprint import
Vanilla HTML5 / CSS3 (`reporter.py`)	Executive triage dashboard (`app_health_report.html`)	Clean dark-mode UI; zero external runtime dependencies
GitHub Actions (Ubuntu runner)	Cloud CI/CD automation & continuous git commit-back	Autonomous weekly cron with state retention

Detailed Tool Analysis:

- `google-play-scraper`**:** Selected over Selenium/Playwright because it queries Google's internal endpoints directly, running in seconds without headless browser overhead or CAPTCHA blockers.
- `sentence-transformers` **(** `all-MiniLM-L6-v2` **):** Selected over commercial embedding APIs (OpenAI `text-embedding-3` , Google Vertex) because it runs 100% locally on CPU. Embedding 1,000 reviews takes ~3.5 seconds with zero API expenses.
- `chromadb` **:** Operates directly inside the Python process memory (`chromadb.Client()`). It provides full HNSW approximate nearest-neighbor search without requiring an external vector database server (such as Pinecone or Milvus).
- **Google GenAI (** `gemini-2.5-flash` **):** Used selectively for edge-case binary decisions, utilizing Gemini 2.5 Flash's fast response times and generous free-tier limits.
- **GitHub Actions:** Provides the complete automation backbone, executing the pipeline on an Ubuntu virtual machine every Sunday at midnight without requiring dedicated infrastructure.

Part 3: Continuous Monitoring Strategy ("Not a One-Time Ask")

To fulfill the assessment's directive that tracking ratings is a continuous requirement, the pipeline is deployed via Infrastructure-as-Code (`.github/workflows/play_store_triage.yml`):

1. **Autonomous Schedule:** Triggers automatically via cron (`0 0 0` — Sunday at midnight UTC).
2. **State Commit-Back Loop:** When new reviews are triaged, GitHub Actions uses `stefanzweifel/git-auto-commit-action` to commit the refreshed `output/rating_trend_history.json` and `output/app_health_report.html` back to the repository (`[skip ci]`).
3. **Engineering Integration:** Automatically prepares `output/mock_jira_tickets.json` for bulk upload via the Jira REST API, while dispatching Slack webhook alerts to mobile engineering channels upon completion.